CPT202 Assignment 2

Group Report for Software Engineering Group Project

2025/2026 Semester 2

Consilium - a booking consultation system

Group number: 22

Submitted by: Jintong Zhang, 2360862

Group Members:

Qi Cao, 2360908
Jintong Zhang, 2360862
Kan Liu, 2362244
Zifan Wang, 2362457
Na Li, 2361132
Yuanhao Cheng, 2362404
Xilai Bu, 2362025
Shengxiang Zhu, 2363335

`http://121.41.36.107/`

May 10, 2026

Contents

I Introduction

A Problem Statement

The professional services industry has moved online, yet customers still face three structural deficiencies. First, specialist discovery is fragmented — customers must search across disconnected websites and portals with no centralized comparison. Second, trust remains opaque — credential verification lacks standardization, leaving customers uncertain about qualifications. Third, the appointment lifecycle is incomplete — many systems handle scheduling in isolation, without integrated status tracking, preparation support, or post-session feedback. These gaps reduce resource utilization, lower user satisfaction, and complicate platform governance.

B Project Aims

To address these challenges, we present **Consilium**, a secure and scalable online consultation booking platform. The project aims to:

- Unify multi-domain specialist discovery with structured, verified profiles.
- Support the complete appointment lifecycle, from booking request to review.
- Build a feedback-driven reputation system to sustain platform quality.

C Project Scope and Core Use Cases

Consilium is precisely positioned as a focused professional consultation booking platform. The system scope do not include online chatting, video conferencing, third-party calendar synchronization, or any native mobile applications;Instead, Consilium has a focused goal centered on delivering a consultation booking service, bounded by the following core use cases:

- **Discover a Specialist** — A customer searches by domain and availability, selects a verified specialist, and submits a booking request for confirmation.
- **Manage an Appointment** — Both parties track the booking from pending through confirmed to completed, with status updates at every transition.
- **Build a Trusted Profile** — A specialist configures availability and credentials; the platform enforces approval before customer exposure and accumulates verified reviews after each completed session.
- **Oversee the Platform** — An administrator monitors accounts, bookings, and announcements, maintaining visibility and control across all activity.

D User Characteristics

User modeling was progressively refined by analysis and trimming. Three roles emerged — each with distinct friction points, distinct workflows, and a distinct relationship with the platform.

- **Customers:** Customers are discovery-oriented, not search-oriented. Customers register on the platform, filter specialists by domain and availability, submit booking requests, track status transitions from pending to completion, and submit post-session reviews. Their primary friction is uncertainty, thus the system must keep them informed through every status change.
- **Specialists:** Specialists are service providers who rely on the platform to manage and convert incoming demand. They configure their profile and availability, receive and act on booking requests, attach meeting details to confirmations, and accumulate verified reputation passively as sessions complete.
- **Administrators:** Administrators are privileged overseers responsible for platform integrity and operational visibility. They approve specialist registrations before customer exposure, manage service categories and announcements, monitor booking activity and user accounts, and maintain an audit trail across all system actions.

E Assumptions, Dependencies, and Risks

Assumptions: All users operate in a single timezone (UTC+8); no multi-timezone handling is required. A single administrator account is sufficient for system management. Specialist qualification certificates are uploaded as JPEG or PNG images, and no document parsing is needed. Users are assumed to have basic web proficiency and familiarity with standard booking terminology such as "Pending," "Confirmed," and "Completed." The expected user base is small to medium scale, so distributed architecture and horizontal scaling are out of scope.

Dependencies: The system is built upon multiple core technologies and third-party services:

- Backend: Spring Boot, Spring Data JPA, Spring Security.
- Frontend: JavaScript, Vite and Lucide.
- MySQL for relational storage, with versioned schema scripts.

Risks: Simultaneous booking requests for the same slot may create conflicts. Mitigation: database-level locking or optimistic concurrency control. Malicious specialists may upload fake certificates, endangering customer safety. Mitigation: mandatory manual review of all uploaded credentials before profile activation.

F First-Release Scope and Prioritization Rationale

Consilium's first release aims to deliver the core booking service, by maintaining a balance across four priority tiers: foundational infrastructure, trust and safety, operational capability, and selective enhancement.

- **Foundation** — Identity and authentication, role-based access control, the end-to-end booking flow, and the underlying data model form the structural core of the system. Establishing this layer first ensures every subsequent feature has a stable, verified base to build upon.
- **Trust and Safety** — Credential governance and verified post-session reviews are prioritised immediately after the foundation because platform credibility depends on them. Specialist vetting and accountable feedback give both customers and specialists sufficient confidence to engage.
- **Operational** — Some functional features like search filtering and consultation feedbacks are included, because they can better facilitate users, make the booking core reliable and governable in practice. Their implementation can better inform all user groups throughout the appointment lifecycle, and the platform remains manageable as activity grows.
- **Enhancement** — Some items are classified as additive features, including real-time communication, analytics, and reminder notifications. They extend the platform but are not mandatory for the core service to function. These are deliberately excluded from the first release and documented as candidates for future implementation.

II Architectural Design

A Architecture Drivers

1 Functional Requirements

This consilium program is a specialists booking system. The core functional requirements directly build the system technique architecture. These functional requirements decide what the system should be separated into different parts, the responsibility border of the different modules and how to flow between different levels.

Role-Based Multi-Actor System: The basic functional driven factor is the role-based multi-actor system. The system needs to support three different roles (Customers, Specialists and Administrators) at the same time. Every role own the completely different functional collection and business process.

Full Booking Management: Full booking management is the second key functional requirements. A complete reservation is from the customer initialization, confirmed or rejected by the expert, the consultation proceeds and the evaluation is completed. This contains several status transfer (**PENDING →CONFIRMED → REJECTED/CANCELLED → CONDUCTED → REVEIWED**). Every status has the program rules' limitations behind, which need the backend service layer to deal with this status change as a transactional way and concentrate all the program checking logic on the service layer.

Multi-Criteria Specialist Search: For the specialist part, the search is the core function targeted as customer. The customers' demand can be selected as category, name and price range in the specialist list. The complexity of the selection logic with multi-table related query (specialists JOIN users JOIN categories) and result pagination processing decide the system must use Spring Data JPA's dynamic query abilities rather than the simple CRUD saving process.

2 Non-Functional Requirements:

Scalability: The system uses Frontend-Backend separation architecture. Spring Boot backend follows the stateless design principle–every authentic messages can be saved in the client JWT token and the

server will not protect any HTTP sessions, which makes the backend can be scaled to multi-non-condition entities. The database layer uses the relative database MySQL, which can offer the scalability to the read-write separation.

Security: Authentication uses the JSON Web Token with no condition token principle [1, 2].The valid duration is made 24h, which make HMAC-SHA algorithm signature. The backend kills all the HTTPs' request using the JwtFilter and complete the analysis, validation and adding the security context in the spring security filter chain, which realizes the identity authentication at the request level. Authorization realizes its function at the service layer–the program will check whether the current role has the operation authority. The frontend saves the JWT token and user roles in local storage and add the token to the request through the Authorization: Bearer <token> in every API called.

Team Parallel Development: The clear layers and module border build a foundation to the time parallel development. The repository of the Spring Data JPA follows the convention over configuration principle. The backend team can develop every independent domain of the business logic (BookingService, SpecialistService and AnnouncementService). The frontend team can develop the UI page through the REST API, using Vite to develop the server API and forward it to the backend for integration testing. The united api.js client layer access the backend interface, which make several frontend developers do the parallel development without any interferences.

B System Architecture

1 Overview of the whole system architecture diagram

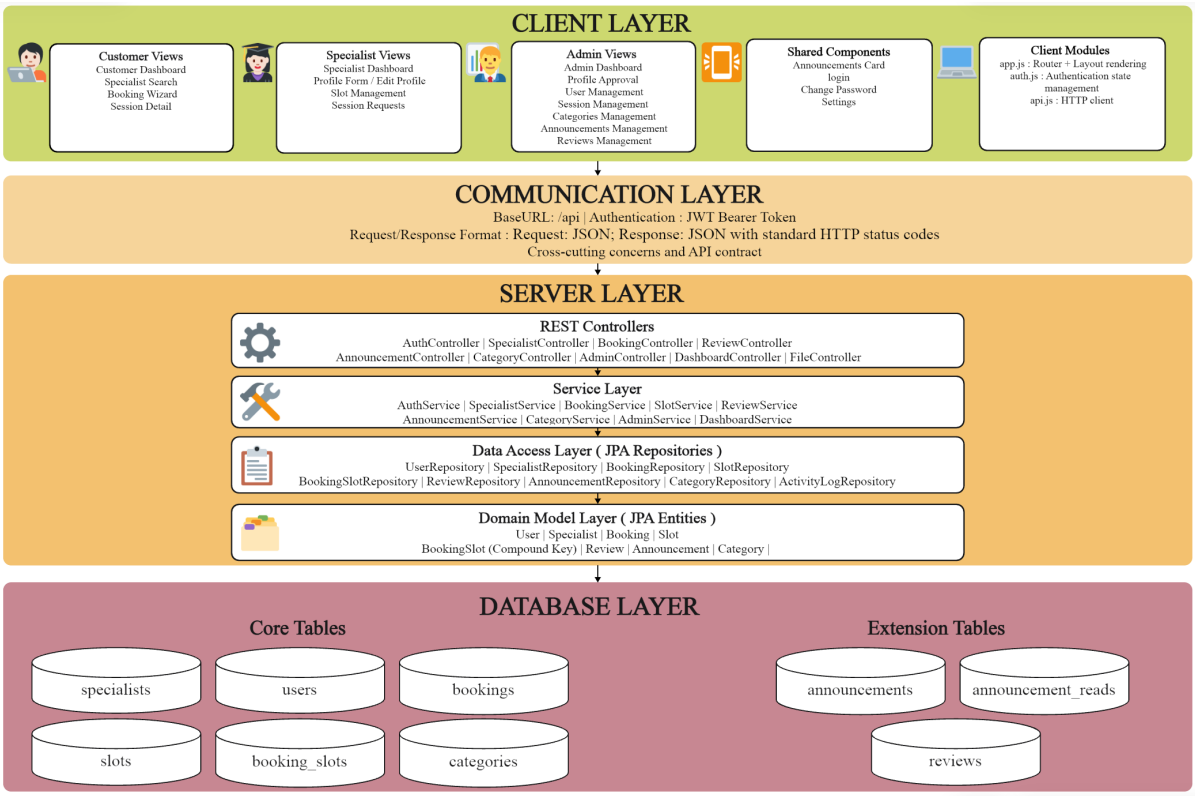


Figure 1: The overall architecture of the system

Figure 1 displays the whole architectural design, each component is clearly structured and has distinct responsibilities.

Client Layer: The client layer (Frontend) is a single page based on the Java Script, organizing the codes under the ES modules without other frontend framework. The frontend data direction: User operations –> Page module operations –> api.js calls the backend REST API –> Backend return JSON –> Page module updates the DOM. During the whole process, the frontend does not maintain the global condition. After page module updating, it calls the API to recover the data.

Communication Layer: The communication layer is responsible for all the cross-system request

protocol including identity authentication token verification, cross-cutting concerns and API contract documentation.

Server Layer: The server layer use Controller-Service-Repository layer architecture embedding in a spring boot single application, accepting HTTP request from the communication layer and executing the business logic to return the response. The REST controllers layer accepts the HTTP request, which has responsibilities to ask the variables' analysis (with DTO + @Valid), the original check the authority and make response to data operations. The service layer is the most important core in the whole system. All the rules, data check and condition movement logic are concentrated in this layer. Every service class adds reliability of other service or repository using the constructor injection.

Database Layer: The system uses MySQL with ORM framework(Spring Data JPA and Hibernate) and DDL strategies (Development environment can automatic synchronization) to save and process the data information. The data flows into the database layer after the server layer making the response.

C High-level Database Design (ERD)

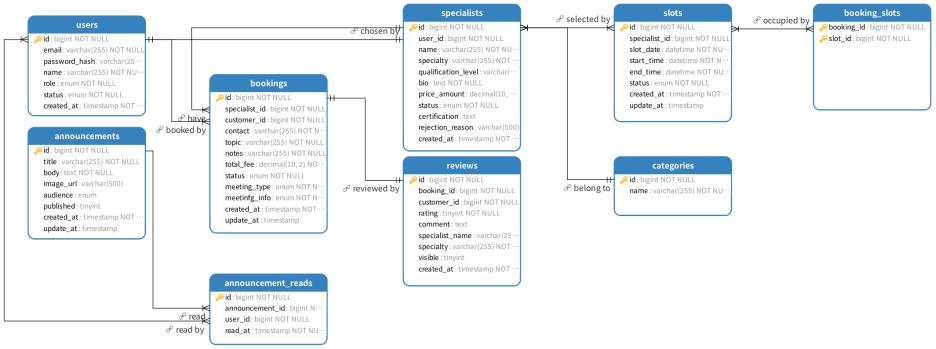


Figure 2: High-level ERD of the Specialist Consultation Booking System

1 Entity Perspective

The database (Figure 2) contains nine main entities, which are designed around the main business objects in the consultation booking process.

- `users`: provides the unified identity layer of the system. Customers, specialists, and administrators are all managed through this table, with different system permissions distinguished by user roles and account status.
- `specialists`: stores the professional service profile of specialist users. It separates specialist-specific business information from general login accounts, allowing profile approval, service status management, qualification information, and consultation pricing to be handled independently.
- `slots`: represent the available time slots created by specialists, which are the basis for availability display, schedule management, and booking conflict prevention.
- `bookings`: records the main appointment information submitted by customers. It connects customers with selected specialists and stores relevant information.
- `booking_slots`: links booking records with selected time slots, which allows the system to record exact time periods.
- `categories`: supports specialist classification and search. It provides a structured category list so that customers can filter specialists more efficiently.
- `reviews`: stores customer feedback after a consultation has been completed. It supports service quality evaluation and allows administrators to control whether reviews are visible to other users.
- `announcements`: is used for platform-level communication. Administrators can publish announcements to different audiences and maintain important system information.
- `announcement_reads`: records whether users have read specific announcements, helping the system track announcement visibility and user engagement.

2 Relationship Perspective

- `users` **to** `specialists` — **One-to-One:** A specialist profile is linked to one user account through `specialists.user_id`. This separates general account information from specialist service profile information.
- `specialists` **to** `slots` — **One-to-Many:** One specialist can create multiple available time slots, while each slot belongs to only one specialist through `slots.specialist_id`.
- `users` **to** `bookings` — **One-to-Many:** One customer can create multiple booking records, while each booking is submitted by one customer through `bookings.customer_id`.
- `specialists` **to** `bookings` — **One-to-Many:** One specialist can receive multiple booking requests, while each booking is associated with one specialist through `bookings.specialist_id`.
- `bookings` **to** `slots` — **Booking-Slot mapping via** `booking_slots`**:** A booking can include multiple slots through `booking_slots`. Although this is implemented with a junction table, business rules additionally enforce that one slot can belong to at most one active booking at a time.
- `bookings` **to** `reviews` — **One-to-One:** Each review is linked to one booking through `reviews.booking_id`, and each booking can have at most one review. This ensures that feedback is tied to an actual consultation record.
- `users` **to** `reviews` — **One-to-Many:** One customer can submit multiple reviews across different bookings, while each review is associated with one customer through `reviews.customer_id`.
- `announcements` **to** `announcement_reads` — **One-to-Many (ID-based):** One announcement can have multiple read records. In implementation, `announcement_reads.announcement_id` stores the referenced announcement ID rather than an object-level `@ManyToOne` mapping.
- `users` **to** `announcement_reads` — **One-to-Many (ID-based):** One user can have multiple announcement read records. In implementation, `announcement_reads.user_id` stores the referenced user ID, with read tracking maintained through ID fields.
- `categories` **to** `specialists` — **One-to-Many :** The `categories` table provides a controlled list of category names for filtering and discovery. The specialist classification is currently matched by text (`specialists.specialty` ↔ `categories.name`), so this is a logical association rather than a database foreign-key constraint.

D Architectural Trade-offs and Support for Incremental Development

The architecture was shaped to enable a smooth path to the first release, allowing parallel work and confident integration, testing, and deployment [5].

1 Support for Incremental Development

- **Modular Vertical Slicing:** The architecture is partitioned by functional domain. Each team member owns a complete vertical slice as shown in Section A, allowing new features to be added by creating new file sets.
- **API-First Contracts:** The team uses PBIs to define API interface contracts before coding begins. This allows the frontend to develop interfaces against mock data before the backend is complete, while the backend implements interface logic independently. During integration, both sides only need to verify contract compliance.
- **Early Definition of Entity Boundaries and Foreign Keys:** Determining entity relationships and foreign keys in advance provides a stable data foundation for each new feature, making sprint planning more predictable.

2 Support for Parallel Team Development

- **Frontend and Backend Parallelism:** The two sub-teams develop in parallel against a shared API specification, using mock data to decouple front-end progress from back-end readiness [5]. The front-end team builds the specialist search page with mock JSON data, while the back-end team implements the `/api/specialists/search` endpoint.
- **Git Branching Strategy:** Each vertical slice is developed on an independent `feature` branch, merging to main via `Pull Request`. The unified `ApiResponse<T>` wrapper at the `Controller` layer and the `DTO` layer provide stable interface boundaries for merging.

3 Support for Integration

- **Consistent Response Format:** All endpoints use the `ApiResponse` envelope format, simplifying the front-end's parsing logic.
- **Early Full-Stack Integration:** The Vite proxy forwards `/api` requests to the local backend, enabling continuous integration to begin early and continuously, avoiding late-stage merge risks.

4 Support for Testing and Deployment

- **Layered, Isolated Testing:** The separation of Controller, Service, and Repository layers allows the service layer logic to be fully unit-tested with Mockito, isolated from the web and database layers.
- **Real HTTP Integration Testing:** Integration tests using Spring Boot Test and MockMvc simulate real HTTP flows passing through security filters, with H2 as the database.
- **JWT Security Testing:** A custom `JwtFilter`, by attaching either valid or invalid JWT tokens, allows authentication and authorization to be tested within the same flow, achieving comprehensive security regression testing without additional manual setup.
- **Repeatable, Immutable Deployment:** A multi-stage Docker build generates the back-end image and the front-end image separately.
- **Service Orchestration and Health Checks:** Docker Compose orchestrates three services within an isolated bridge network, using health checks to ensure MySQL starts before other services.
- **Single-Origin CORS Simplification:** An Nginx reverse proxy serves the SPA and `/api` routes under the same origin, eliminating cross-origin issues.
- **One-Click Consistent Environment:** The command `docker compose up -d` starts the entire technology stack, ensuring that the evaluation environment is completely consistent with the development environment.

5 Major Architectural Trade-offs

The architecture trades off scalability reserves for simplicity, team velocity, and a repeatable pipeline—appropriate for a small-scale first release.

- **Monolithic backend vs. microservices.** A single Spring Boot application avoids the overhead of service discovery, distributed transactions, and per-service pipelines. The cost is that modules cannot be scaled or deployed independently; however, the stateless design and containerization keep future decomposition feasible.
- **Frontend–backend separation vs. server-side rendering.** A JavaScript SPA communicates exclusively through REST APIs, enabling parallel development on mock contracts and single-origin serving via Nginx without CORS issues. The cost is that the SPA must implement token storage, routing, and UI updates without a framework's built-in support.
- **Stateless JWT vs. server-side sessions.** JWTs (24h validity) validated on every request keep the backend stateless, easing horizontal scaling and integration testing. The trade-off is that instant token revocation is not available; a block-list can be added later if required.
- **Relational model with strict foreign keys vs. NoSQL or eventual consistency.** MySQL enforces referential integrity and transactional logic, guaranteeing strong consistency for the core booking workflow. The trade-off is limited write scalability; a shift to CQRS or eventual consistency would be needed for geographically distributed deployments.

III Software Design and Engineering Practice

A Software Modules and Responsibilities

The Consuilium system uses front-backend decoupling architecture which is divided into 8 different modules. The backend is divided based on the functional module. The frontend is divided based on the role page contents. The border of the modules is consistent with the division of labor within the team. Figure 3 shows how these modules are connected and share data.

- **M1: User Auth & Access Control Module** - Makes registry/login/logout, JWT tokens, Manages roles and Password security.

- **M2: Specialist Profile Management Module** - Manages specialists profiles CRUD (name, domain, bio, contact information), Clients checking, moderate/suspend the role, credential uploads, bio updates and approval tracking.
- **M3: Availability & Schedule Management Module** - Sets specialists available time slots, defence the repetitive booking and automatically close after the time slots booked (AVAILABLE → BOOKED)
- **M4: Booking & Appointment Flow Module** - Chooses specialists and time slots, submits the booking request, seeing fees, confirms/declines the reservation, core transaction write path
- **M5: Search & Discovery Module** - Searches the specialists, designs the entry of the booking flow. This improves matching efficiency.
- **M6: Booking Status & Lifecycle Module** - Manages status changes (Pending → Confirmed/Rejected → Conducted → Reviewed/Cancelled), It also handles status requests and triggers notifications.
- **M7: Client Dashboard & History Module** - Displays pending/history reservation containing status label and cancelling entrance, reads intensively (data comes from M4 and M6)
- **M8: Admin Panel Module** - User activate/suspend, monitoring all the reservations and handling dispute, cuts through all the modules, offering visibility and controls at system level.

The modularization design makes a clear border and highly efficient collaborative mode. Through several clear modules, every module corresponds to the specific functional sets and code library, which makes the team member can develop independently, test and deploy in their respective areas of responsibility. It basically reduce the conflict errors due to the parallel development.

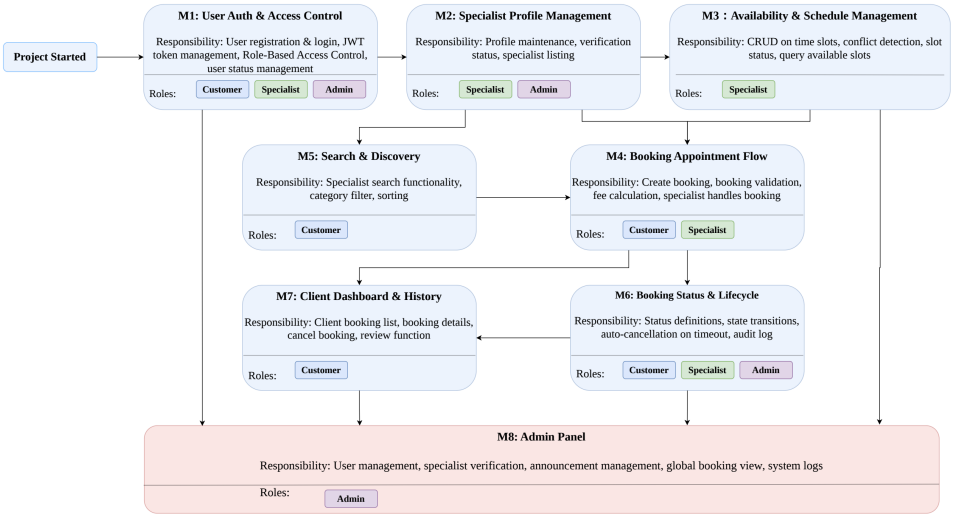


Figure 3: Module Relationship Diagram

B High-Level Process Flows of Core Functions

Based on the software modules defined, our interactions through three core workflows:

1. User Authentication and Administration
2. Specialist Management
3. Booking and Appointment Flow

1 User Authentication and Administration

This flow covers the complete user registration, login, and administration lifecycle. It involves two core modules: M1 (User Authentication and Access Control) as the global security infrastructure, and M8 (Admin Panel Module) for administrator oversight.

Registration Flow: Once the user submits the registration form, M1 module let AuthController triggers DTO validation via @Valid. AuthService.register() then proceeds through the following steps—hashing the password with BCrypt (which embeds a per-hash salt inline, effectively defeating rainbow-table attacks); persisting the user through userRepository.save(); additionally creating a specialist record with status = PENDING if the role is SPECIALIST; and invoking activityLogService.log() to write an audit entry. As shown in Figure 19.

Login Flow: M1 is responsible for the AuthService.login() to retrieve the user via userRepository.findByEmail(), compare the password with BCrypt, and then call jwtUtil.generateToken() to issue a JWT (the payload carrying email and role, signed with HMAC-SHA512). The front end stores the token in localStorage. All subsequent requests automatically carry the Authorization: Bearer header, which JwtFilter intercepts and validates before populating the SecurityContext, as shown in Figure 22. The detailed login sequence is shown in Figure 20.

Administration supervision: The JWT-based stateless approach ensures seamless session management across distributed system components. M8 collaborates with M1 to let administrators manage user accounts and roles. The logout process clears the token from localStorage and invalidates the session context, as shown in Figure 21. Administrators use the admin panel to review and approve specialist profiles submitted through M2, update user account status via M1, and manage announcements targeted at specific user groups. The APM provides visibility into all bookings through M6, monitors review content through M4 to maintain platform integrity.

2 Specialist Management

This flow covers the complete specialist lifecycle from initial registration to active practice and profile maintenance. It involves three modules: M2 (Specialist Profile Management), M3 (Slot Availability Management), and M8 (Admin Panel Module).

Registration and Review: Stage 1: Entering the review queue upon registration (M2). When a specialist registers, AuthService (M1) creates a specialist record with status = PENDING, which is not externally visible. Stage 2: Administrator review. The administrator sends PATCH /api/admin/specialists/profile/{id}/review, and AdminService.reviewSpecialistProfile() sets specialist.status = ACTIVE (for APPROVE) or status = REJECTED (for REJECT) according to the action, with an accompanying audit log entry. Stage 3: (M8) Automatic re-review on profile change (key design). When an already-approved specialist modifies substantive fields (area of expertise, rate, or credentials), SpecialistService.updateProfile() automatically resets status to PENDING, sparing the specialist any need to re-apply manually.

Profile and Schedule Coupling: In M2, once a specialist is approved (status = ACTIVE), they can create available time slots. And M3, the specialist submits filter criteria to set up their schedule, and SlotService queries the slots table filtered by specialist_id, slot_date, and status = AVAILABLE. Slot overlap detection prevents creating multiple slots for the same time window. M2 manages specialist profile data and approval status. M3 depends on M2 — only ACTIVE specialists can create slots, and only ACTIVE specialists appear in client searches. M8 oversees both: it controls the approval/rejection workflow in M2 and monitors specialist availability through M3.

3 Booking and Appointment Flow

This flow covers the complete booking lifecycle from discovery to review. It involves six modules: M1 (Authentication), M3 (Slot Management), M4 (Booking Flow), M5 (Search & Discovery), M6 (Booking Status Lifecycle), and M7 (Client Dashboard).

Discover a Specialist: In M5, once the client submits filter criteria, SpecialistService.search() performs cascading queries through categoryRepository and specialistRepository, joining the users, specialists, and categories tables and returning a paginated result set. Only specialists with status = ACTIVE are returned in search results in M2.The specialist search and public profile browsing sequence is shown in Figure 23.

View Availability: In M3 part, The front end requests /api/specialists/{id}/schedule_date=YYYY-MM-DD, and SlotService queries the slots table filtered by specialist_id, slot_date, and status = AVAILABLE. Clients can only select slots that are in the AVAILABLE state.

Submit the Booking: The front end sends a BookingCreateRequest, and BookingService.createBooking() performs five core checks: Authorization check: verify that currentUser.role == CLIENT (M1 provides the authenticated context through JwtFilter and SecurityContext). Slot availability check: slotRepository.findByIdIn(slotIds) verifies one by one that each slot is in the AVAILABLE

state (M3). Conflict detection: bookingRepository.existsBySlotIdInAndStatusIn(slotIds, [CONFIRMED, CONDUCTED, REVIEWED]) prevents double-booking of the same time window. Fee calculation: fee = specialist.basePrice + slotIds.size × specialist.hourlyRate. @Transactional write: create the Booking (status = PENDING) → persist the booking_slots associations → write the audit log → COMMIT.The booking submission sequence is shown in Figure 24, and the specialist-side status transition and booking response process is shown in Figure 25.

Dashboard and Review: After the consultation is conducted, the client can view their booking history through M7's dashboard. The booking completion sequence is shown in Figure 27. ReviewService.submitReview() performs four checks: booking existence → booking status must be CONDUCTED → review uniqueness (one per booking) → authorization (only the booking's own client may submit). Once the checks pass, a Review is created within the transaction (visible = false, i.e., not public by default), booking.status is updated to REVIEWED, and an audit log entry is written. After administrator moderation sets visible to true, the review becomes publicly accessible through GET /api/reviews/recent. This design decouples submission from publication, thereby safeguarding content quality on the platform. The client-side booking cancellation sequence is shown in Figure 26.

C Support Services and Engineering Tools

1 Database Services

Relational Database Integration: The system uses **MySQL 8.0+** for persistent storage of core business data, including user profiles, specialist profiles, and slots. The backend interacts with MySQL via **Spring Data JPA** and **Hibernate** ORM, which abstract complex SQL queries and provide a robust repository layer [2]. Configuration is managed in `application.properties` using environment variables for secure credential injection.

Connection Pool Management: HikariCP (bundled with Spring Boot) manages the database connection pool, configured for optimal performance through connection timeout and pool size settings. **Spring Boot**'s auto-configuration ensures efficient resource utilization under high concurrency, enhancing performance during heavy loads.

2 Security Services

Authentication and Authorization Framework: Spring Security is integrated to enforce role-based (User, Admin and Specialist) access control. User passwords are hashed using **BCryptPasswordEncoder** before storage. Security filter chains are configured to protect API endpoints, with public access restricted to login, registration, and static resources.

Stateless Authentication with JWT: We employ **JJWT (0.12.6)** for stateless API authentication. Upon successful login, the backend issues a signed JWT containing user roles and expiration. The frontend stores this token and attaches it to the `Authorization: Bearer <token>` header via **Axios** interceptors. A custom `JwtFilter` validates tokens on each request, ensuring secure and scalable session management.

3 Development Tools and Engineering Practices

Our development process was supported by a set of modern tools and practices that facilitated collaboration, ensured code quality, and simplified deployment. Figure 4 illustrates the development tools used in our system.

- **Version Control & Collaboration:** All source code was managed with **Git** and hosted on **GitHub**. The team adopted a branching strategy (Figure 17) where feature development was carried out on dedicated branches, and all code merged into the main line through **Pull Requests** (Figure 18) with mandatory peer review to ensure knowledge sharing and quality control.
- **Build and Dependency Management:** Backend dependencies and the project lifecycle were managed with **Apache Maven**. A shared `pom.xml` file guaranteed consistent library versions across all development environments. The frontend utilized **Vite** as its build tool, providing fast module bundling and a smooth development experience.
- **Frontend Tooling:** The user interface was implemented using **JavaScript** without a heavy framework, with the native `fetch` API handling all asynchronous communication with the backend. A **custom hash-based router** managed client-side navigation, and a centralized module handled authentication headers and error interception to maintain a clean separation of concerns.

- **Containerization & Deployment:** The entire system was containerized using **Docker** and orchestrated with **Docker Compose**. Multi-stage builds were employed to create lightweight, production-optimized images for both the backend service and the Nginx-served frontend. This approach ensured a reproducible environment from development to production, enabling one-command startup for the full stack.
- **Developer Productivity: Lombok** was used in the Java backend to reduce boilerplate code, while **Spring Boot DevTools** enabled hot reloading for rapid iteration during development. Consistent coding standards were enforced across the team, with the backend following standard Java naming conventions and the frontend adhering to modern JavaScript best practices, all maintained through shared IDE configuration files.

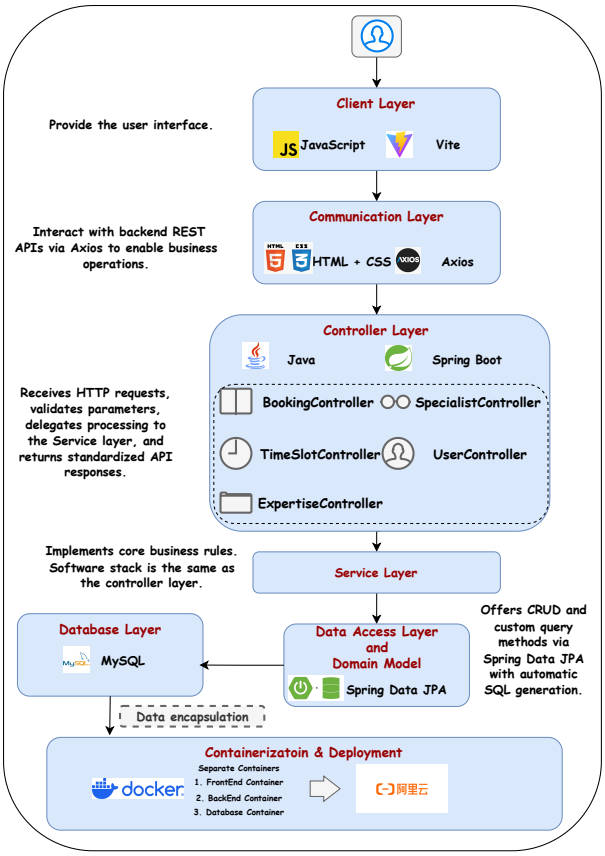


Figure 4: System Architecture with development tools

D Coding Structure and Convention

1 Coding Structure

As Figure 5 shown, the system adopts a layered architecture with a clear separation between the frontend and backend components. The backend is developed using the Spring Boot framework and follows the Model–View–Controller (MVC) architectural pattern, while the frontend is implemented using HTML, CSS, and JavaScript [3, 5].

This design promotes modularity, maintainability, and role-based extensibility.

2 Coding Convention

In our project, we established a unified coding style and development guidelines, which improved team collaboration and enhanced the readability and maintainability of the codebase.

- **Identifier Naming:** Variables, methods, and parameters follow the lower camelCase convention (e.g., `findUserById`), while classes and interfaces use Upper CamelCase (e.g., `BookingController`). Database fields follow the snake_case naming convention (e.g., `specialist_id`).

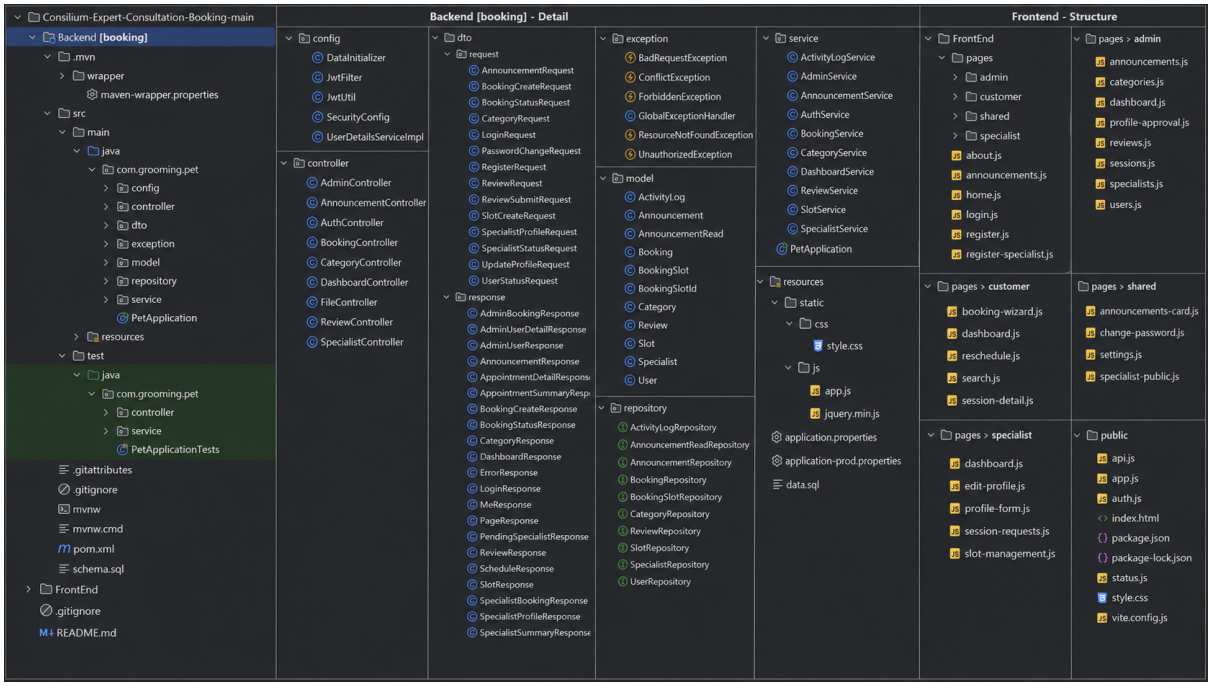


Figure 5: Code architecture overview of the backend and frontend modules.

Table 1: Production Server Configuration

Specification	Value
Operating System	Ubuntu 22.04 LTS
CPU & Memory	2 vCPU, 4 GiB
Public Bandwidth	2 Mbps

- **Error Handling:** The system implements unified error handling mechanisms on both the frontend and backend. The backend adopts centralized exception handling via @RestControllerAdvice and maps common exceptions to standardized HTTP responses. Business validation errors are handled in service logic, while selective try-catch is used for input parsing and specific failure cases. The frontend handles asynchronous API requests using async/await with try-catch (and .catch()), and provides user-friendly feedback through toast messages and inline error prompts to improve robustness.
- **Branching and Commit Convention:** Our team used a feature-branch workflow with Git and GitHub for collaborative development. Each member worked on separate branches, and changes were merged through Pull Requests (PRs). Consistent commit message formats such as `feat:` and `fix:` were adopted to improve code management and traceability.

E Production Environment and Deployment Approach

1 Production Environment

Our system is constructed using Java 17 and Spring Boot 3.4.5 for the backend, JavaScript ES2020+, Vite, Axios, and Lucide for the frontend, MySQL 8.0+ for database management, Nginx as the web server and reverse proxy, and Docker for containerized deployment.

The system is deployed on an **Alibaba Cloud Elastic Compute Service (ECS)** instance to ensure high availability and public network access. The configuration of the production server is shown in Table 1.

2 Deployment Approach for First Release

The deployment strategy prioritizes consistency and immutability to meet the requirement that the software at the submission URL remains unchanged for one week post-submission.

- **Containerized Deployment with Docker Compose:** The entire application stack is deployed as Docker containers, orchestrated by Docker Compose:
  1. **Image Building:** Docker builds the backend image (multi-stage: Maven compile → JRE runtime) and frontend image (multi-stage: npm ci + vite build → Nginx static serve) using `docker compose build` from the `docker/` directory.
  2. **Service Orchestration:** Docker Compose starts three containers on an isolated bridge network (`consult-network`) — MySQL 8.0 (`consult-mysql`), Spring Boot backend (`consult-backend`), and Nginx frontend (`consult-frontend`). Containers communicate by service name via Docker's internal DNS.
  3. **Health-Check Dependency:** The backend container waits for MySQL to pass its health check (`mysqladmin ping` every 10s, up to ∼100s timeout via `start_period: 30s` and `retries: 10`) before starting, preventing database race conditions.
  4. **One-Command Deployment:** The entire stack is deployed with a single `docker compose up -d` command from the `docker/` directory, ensuring immutability and reproducibility across all environments.
  5. **Data Persistence:** MySQL data is stored in a named Docker volume (`mysql-data`) mapped to `/var/lib/mysql`, so database state survives container restarts and is only removed explicitly via `docker compose down -v`.
- **Reverse Proxy Configuration: Nginx** (inside the `consult-frontend` container) acts as the unified entry point. Requests to the root path (`/`) serve the static files with SPA fallback routing, while requests beginning with `/api/` are proxied to the backend service via Docker's internal DNS name `http://backend:8080`. This internal communication stays within the Docker bridge network, eliminating unnecessary network exposure.
- **Stability Guarantee for Submission:**
  - **Frozen Release:** The codebase for submission has been frozen and will not be modified during the evaluation period.
  - **Monitoring:** Basic server monitoring ensures the application remains accessible during the evaluation period.

F Legal, Social, Ethical, and Professional Considerations

Legal: Consilium operates with personal data, professional credentials, and user reviews — each carrying distinct legal obligations. Users must be informed of what is collected and why; specialists bear legal responsibility for the authenticity of the credentials they submit for approval; and clients are accountable for the accuracy and fairness of the reviews they publish.

Social: Social considerations involve the platform's broader inclusivity and societal impact. Consilium ensured that interface can accommodate varying technical literacy, prevented discriminatory gatekeeping during specialist approvals, and maintain basic accessibility standards to prevent the exclusion of users with disabilities.

Ethical: Ethical concerns impacts the fairness of system design. This requires preventing hidden biases in search rankings, ensuring the administrator's centralized power over specialist accounts, and restricting reviews to verified bookings to prevent reputational sabotage.

Professional: Professional considerations reflect engineering integrity. We demonstrated this by strictly adhering to industrial standards (e.g., JWT, BCrypt), enforcing rigorous peer code reviews, and responsibly documenting and excluding incomplete features rather than deploying unverified code.

IV Software Testing

The testing work for Consilium follows a risk-driven and layered strategy, covering service-level business rules, backend API integration, security-related regression scenarios, and user-oriented acceptance validation. As shown in Figure 6, the testing process starts from risk and requirement analysis, then proceeds through unit test design, integration test design, test data setup, Maven execution, and evidence-based review.

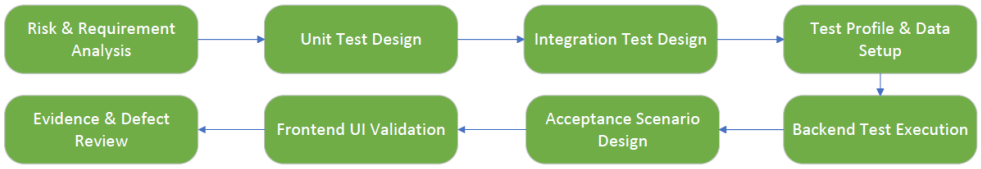


Figure 6: Consilium software testing workflow.

```
@Test
void changePassword_shouldSaveNewPassword() {
    User user = user( id: 1L,  email: "client@example.com", User.Role.CLIENT);
    authenticateAs( email: "client@example.com");
    when(userRepository.findByEmail("client@example.com")).thenReturn(Optional.of(user));
    when(passwordEncoder.matches( rawPassword: "Password123",  encodedPassword: "hash")).thenReturn( t: true);
    when(passwordEncoder.encode( rawPassword: "Password456")).thenReturn( t: "new-hash");

    authService.changePassword(passwordRequest( oldPassword: "Password123",  newPassword: "Password456"));

    assertEquals( expected: "new-hash", user.getPasswordHash());
    verify(userRepository).save(user);
}
```

Figure 7: Unit test evidence for authentication-related service logic.

A Unit Testing

Unit testing was used to verify whether the core service-layer business rules of Consilium behave as expected. The tests focus on service logic related to authentication, booking, slot scheduling, category management, and administration. JUnit 5 provides the test structure, while Mockito is used to simulate repository dependencies and external collaborators. This allows the service layer to be tested without starting the full application stack or relying on the real MySQL database [4].

Test Case 1: Authentication Service Logic

As shown in Figure 7, the authentication-related unit tests verify registration, login, profile update, and password change logic. Mockito is used to simulate repository access, password handling, and token generation, allowing the service layer to be tested without the real database or web layer. These tests cover both successful cases and invalid cases, such as duplicated email, incorrect credentials, password reuse, and unauthorized operations.

Test Case 2: Booking Service Logic

As shown in Figure 8, the booking-related unit tests verify appointment creation and status update rules. Mockito is used to simulate booking, specialist, slot, and repository dependencies, so the service layer can check valid bookings, unavailable slot rejection, and status transitions in isolation. These tests are important because booking is the core workflow of Consilium, where role access, slot conflicts, and illegal state changes must be handled correctly.

The results are shown in Appendix G. These tests do not directly prove that the whole system works end to end, but they confirm that the core business rules are verifiable at the service layer. By testing authentication and booking as two key modules, the project can identify issues related to role permissions, input validity, and status transitions at an early stage, thereby reducing the risk of later API integration

```
@Test
void createBooking_shouldReserveSlot() {
    User client = user( id: 1L, User.Role.CLIENT);
    Specialist specialist = specialist( id: 2L, user( id: 2L, User.Role.SPECIALIST));
    Slot slot = slot( id: 10L, specialist, Slot.Status.AVAILABLE);
    when(authService.getCurrentUser()).thenReturn(client);
    when(slotRepository.findById(10L)).thenReturn(Optional.of(slot));
    when(bookingSlotRepository.hasActiveBooking( slotId: 10L)).thenReturn( t: false);
    when(bookingRepository.save(any(Booking.class))).thenAnswer(invocation -> {
        Booking booking = invocation.getArgument( i: 0);
        booking.setId(99L);
        return booking;
    });

    BookingCreateResponse response = bookingService.createBooking(createRequest(List.of( e1: 10L)));

    assertEquals( expected: 99L, response.getBookingId());
    assertEquals( expected: "PENDING", response.getStatus());
    assertEquals(new BigDecimal( val: "120.00"), response.getFee());
    assertEquals(Slot.Status.UNAVAILABLE, slot.getStatus());
}
```

Figure 8: Unit test evidence for booking-related service logic.

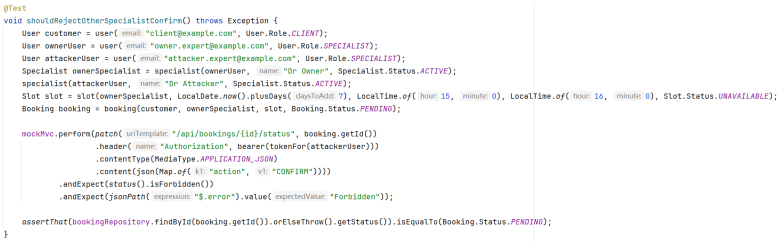
```
@Test
void shouldRejectDoubleBooking() throws Exception {
    User first = user( email: "first@example.com", User.Role.CLIENT);
    User second = user( email: "second@example.com", User.Role.CLIENT);
    Specialist specialist = specialist(user( email: "expert@example.com", User.Role.SPECIALIST),  name: "Dr Race", Specialist.Status.ACTIVE);
    Slot slot = slot(specialist, LocalDate.now().plusDays( daysToAdd: 8), LocalTime.of( hour: 9,  minute: 0), LocalTime.of( hour: 10,  minute: 0), Slot.Status.AVAILABLE);

    mockMvc.perform(post( urlTemplate: "/api/bookings")
                    .header( name: "Authorization", bearer(tokenFor(first)))
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(json(Map.of(
                            k1: "slotIds", List.of(slot.getId()),
                            k2: "contact", first.getEmail(),
                            k3: "topic",  v3: "Commercial readiness"))))
            .andExpect(status().isCreated())
            .andExpect(jsonPath( expression: "$.status").value( expectedValue: "PENDING"));

    mockMvc.perform(post( urlTemplate: "/api/bookings")
                    .header( name: "Authorization", bearer(tokenFor(second)))
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(json(Map.of(
                            k1: "slotIds", List.of(slot.getId()),
                            k2: "contact", second.getEmail(),
                            k3: "topic",  v3: "Commercial readiness"))))
            .andExpect(status().isConflict())
            .andExpect(jsonPath( expression: "$.error").value( expectedValue: "Conflict"));

    assertThat(bookingSlotRepository.findBySlot(slot)).hasSize( expected: 1);
}
```

Figure 9: Integration test code for booking slot conflict handling



```
@Test
void shouldRejectOtherSpecialistConfirm() throws Exception {
    User customer = user( email: "client@example.com", User.Role.CLIENT);
    User ownerUser = user( email: "owner.expert@example.com", User.Role.SPECIALIST);
    User attackerUser = user( email: "attacker.expert@example.com", User.Role.SPECIALIST);
    Specialist ownerSpecialist = specialist(ownerUser,  name: "Dr Owner", Specialist.Status.ACTIVE);
    specialist(attackerUser,  name: "Dr Attacker", Specialist.Status.ACTIVE);
    Slot slot = slot(ownerSpecialist, LocalDate.now().plusDays( daysToAdd: 7), LocalTime.of( hour: 15,  minute: 0), LocalTime.of( hour: 16,  minute: 0), Slot.Status.UNAVAILABLE);
    Booking booking = booking(customer, ownerSpecialist, slot, Booking.Status.PENDING);

    mockMvc.perform(patch( urlTemplate: "/api/bookings/{id}/status", booking.getId())
                    .header( name: "Authorization", bearer(tokenFor(attackerUser)))
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(json(Map.of( k1: "action",  v1: "CONFIRM"))))
            .andExpect(status().isForbidden())
            .andExpect(jsonPath( expression: "$.error").value( expectedValue: "Forbidden"));

    assertThat(bookingRepository.findById(booking.getId()).orElseThrow().getStatus()).isEqualTo(Booking.Status.PENDING);
}
```

Figure 10: Integration test code for booking status update access control

and acceptance testing.

B Integration Testing

Integration testing was used to verify whether different backend layers work together correctly after individual service logic had been tested in isolation. In this project, Spring Boot Test, MockMvc, Spring Security Test, and the H2 test database were used to simulate API requests in a controlled test environment. Unlike unit tests, integration tests send HTTP-like requests through MockMvc and allow the request to pass through the security filter, controller, service layer, repository/JPA layer, and the test database. Therefore, integration tests can check not only the response returned by an endpoint, but also the cooperation between authentication, authorization, business logic, and persistence [4, 5].

Test Case 1: Booking Slot Conflict Handling

As shown in Figure 9, this test checks that two customers cannot book the same available slot. The first booking request succeeds through `/api/bookings`, while the second request for the same slot returns `409 Conflict`. This verifies the slot conflict rule across the controller, service, and persistence layers.

Test Case 2: Booking Status Update Access Control

As shown in Figure 10, this test checks that an unrelated specialist cannot confirm another specialist's booking. The request to `/api/bookings/{id}/status` returns `403 Forbidden`, and the booking remains `PENDING`. This verifies both access control and protection against unauthorized state changes.

C Acceptance Testing

Acceptance testing is a crucial phase in the software development lifecycle in which the system is evaluated to determine whether it satisfies the specified requirements and is ready for delivery. The test cases were designed based on the PBIs and user requirements. Testing was conducted through the application UI within a testing environment, and the actual results were compared with the expected results defined in the PBIs.

The acceptance testing covers the major functional requirements and key system features specified in the project requirements, including:

- **User Registration and Login:** Users can register and log into the system as either customers or specialists.

- **Specialist Profile Management:** Specialists can edit their profiles, including selecting service categories, setting consultation prices, and updating personal biographies.
- **Specialist Booking:** Customers can view specialists' available time slots and make bookings online.
- **Specialist Discovery:** Customers can browse and view detailed information about specialists.
- **Administrator Capabilities:** Administrators can manage the status and information of specialists and customers, as well as publish system announcements.

The following case is an example of acceptance testing for the **Specialist Booking function**:

Acceptance Test Case 1: Specialist Booking

Test Steps:

1. Open the `Find Specialists` page.
2. Select the target specialist and click the specialist card to choose an available time slot.
3. Click the `Go to Booking` button.
4. Fill in the contact details and other required information.
5. Click the `Confirm Booking` button.

Expected Results:

1. The specialist page should be displayed correctly. The user should be able to view the specialist's available time slots.
2. The estimated consultation fee should be displayed correctly.
3. The booking page should be displayed correctly.
4. The system should display a success or failure notification after the booking is submitted.

The screenshots of testing results are in Figure 12. Other screenshots of acceptance testing are demonstrated in the Appendix C. The testing results demonstrate that the system functions correctly and is capable of meeting the core requirements and expectations of end users.

D Defect Tracking and Improvement Before First Release

As shown in Appendix F, testing was embedded into the development workflow from the earliest sprints. Before each new sprint, the team verified the previously implemented features to guarantee a stable, runnable baseline.

When a defect or inconsistency was discovered, the team first analysed its root cause, identified the affected module, and assigned the issue to the responsible developer. Fixes were developed on separate feature branches, merged through pull requests after peer review, and immediately re-tested to confirm resolution and prevent regression. Detailed examples are shown in the Appendix H.

These defect driven improvements enhanced the stability, robustness, and maintainability of the system before delivering the first release.

V References

References

[1] S. Mamidi, "Building Secure REST APIs in Spring Boot: Techniques and Tools," *International Journal of Emerging Trends in Computer Science and Information Technology*, vol. 7, no. 1, pp. 87–91, 2026.

[2] A. J. Sierra Collado and A. Monago Ruiz, "Servicio Web API REST sobre el Framework Spring, Hibernate, JSON Web Token y BBDD Oracle," Universidad de Sevilla, Departamento de Ingeniería Telemática, 2019.

[3] A. Majeed and I. Rauf, "MVC Architecture: A Detailed Insight to the Modern Web Applications Development," *Peer Review Journal of Solar & Photoenergy Systems*, vol. 1, no. 1, pp. 1–7, 2018.

[4] K. El-Morabea and H. El-Garem, "Testing Pyramid," in *Modularizing Legacy Projects Using TDD: Test-Driven Development with XCTest for iOS*. Cham, Switzerland: Springer, 2021, pp. 65–83.

[5] M. Broy and B. Selić, "Specifying and Composing Layered Architectures," *Journal of Object Technology*, vol. 23, no. 1, pp. 1–24, 2024, doi: 10.5381/JOT.2024.23.1.A2.

VI Appendix

A Testing Accounts

- Admin account: username = admin@gmail.com, password = Password123
- Specialist account: username = Priya.Sharma@gmail.com, password = Password123
- Customer account: username = Alice@gmail.com , password = Password123

B Use Case Diagram

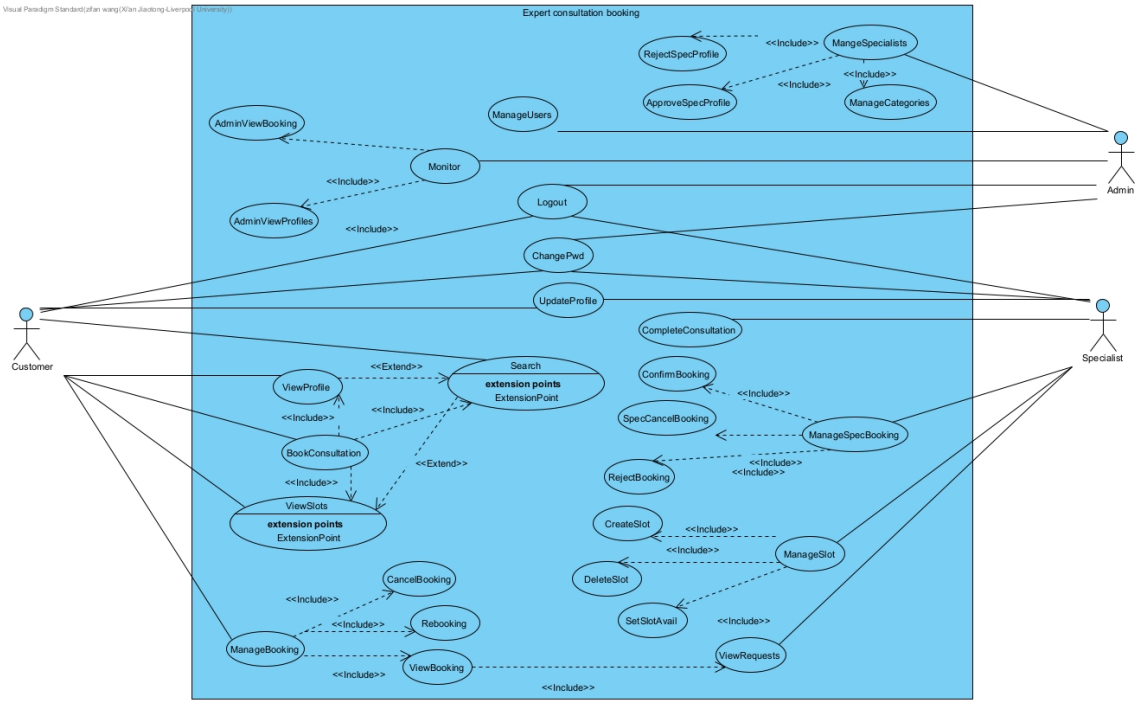


Figure 11: Use case diagram of the Consilium system

C Acceptance Testing Screenshots

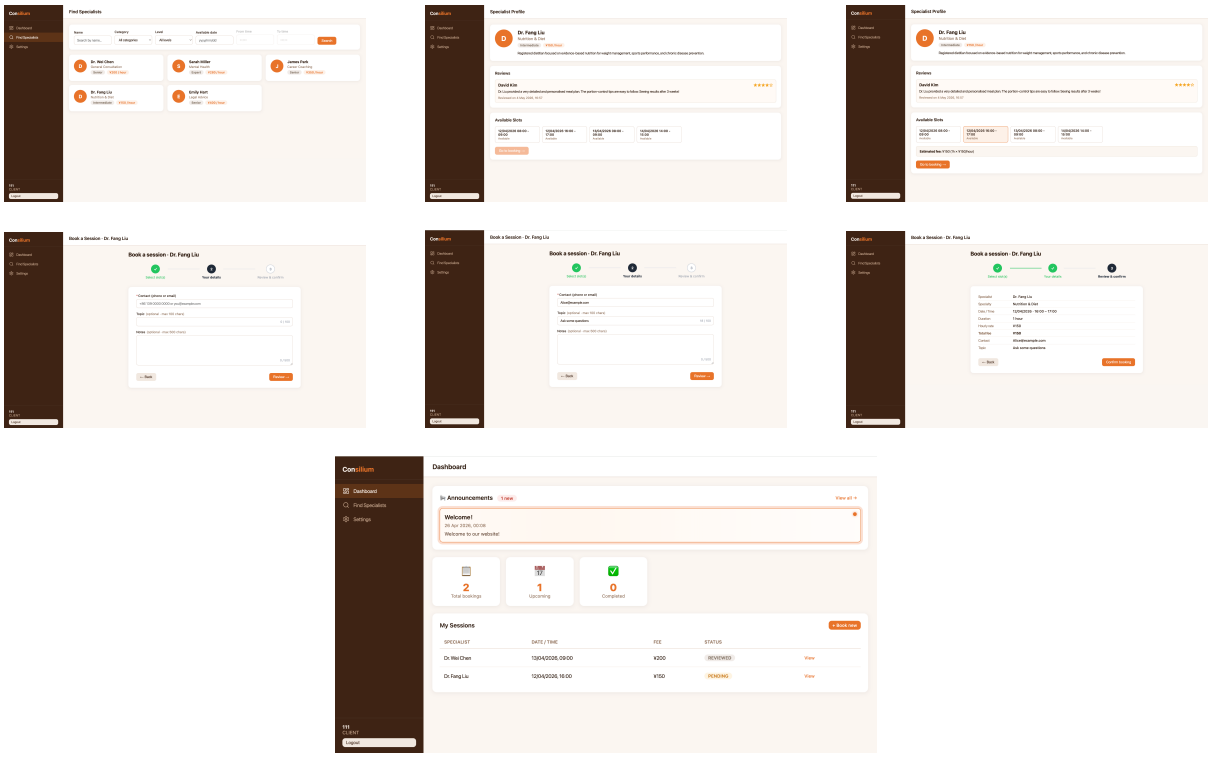


Figure 12: Acceptance Testing for the Specialist Booking.

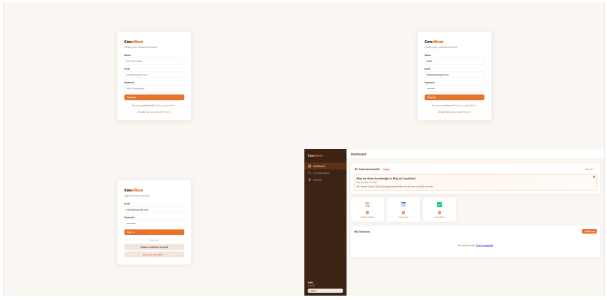


Figure 13: Acceptance Testing for the User Registration and Login.

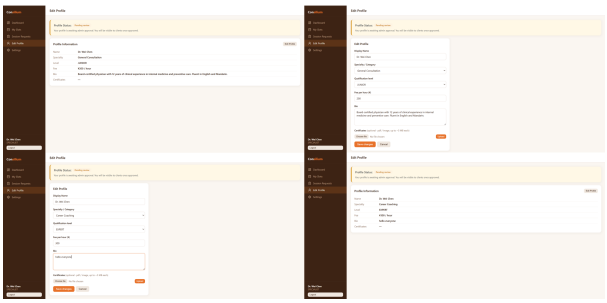


Figure 14: Acceptance Testing for the Specialist Profile Management.

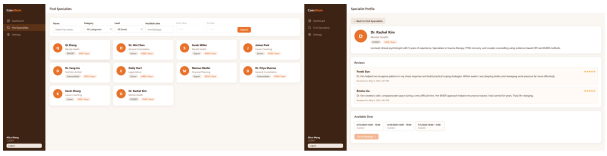


Figure 15: Acceptance Testing for the Specialist Discovery.

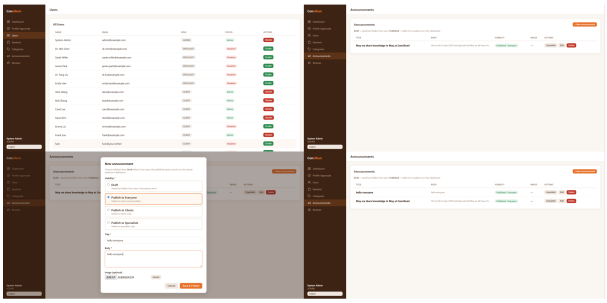


Figure 16: Acceptance Testing for the Administrator Capabilities.

D Collaboration Evidence

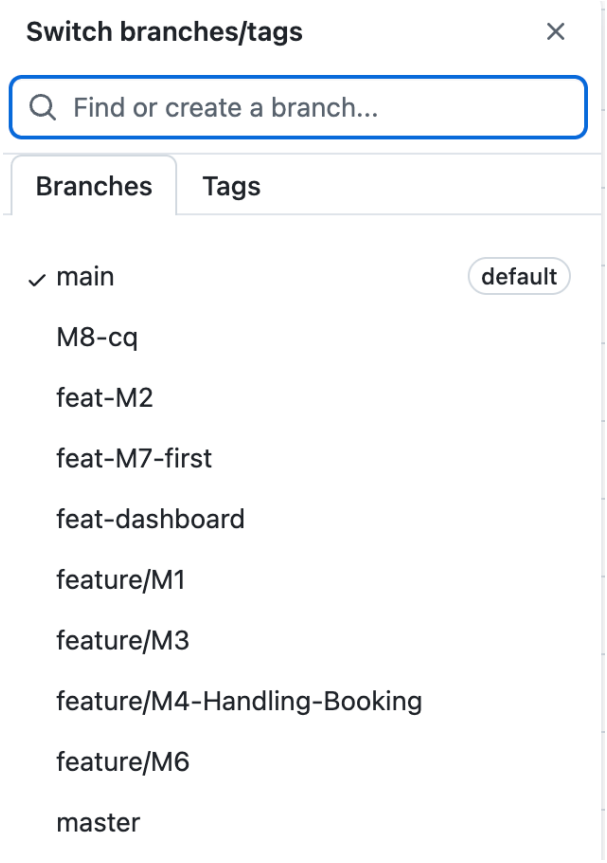


Figure 17: GitHub branches used for collaborative development and feature-based task management.

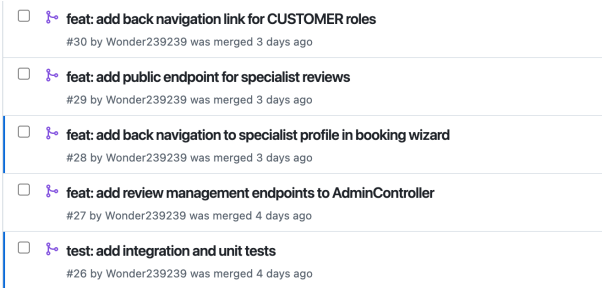


Figure 18: GitHub pull request records showing collaborative development workflow and code integration process.

E Sequence Diagrams of Core Functions

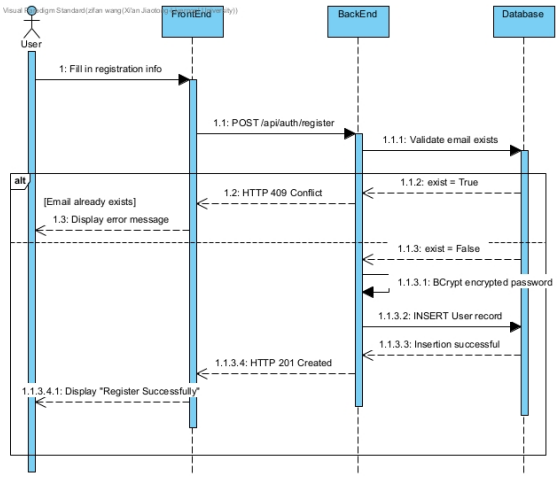


Figure 19: Register

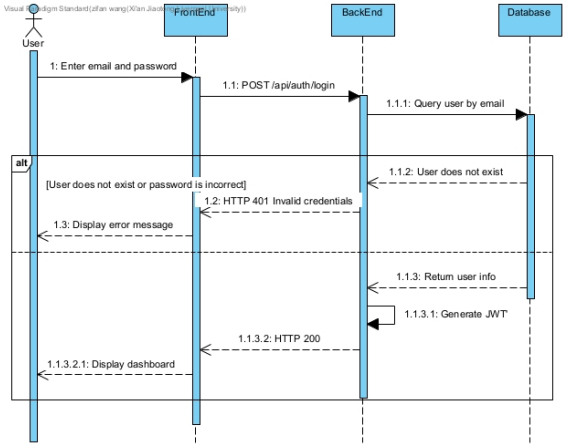


Figure 20: Caption

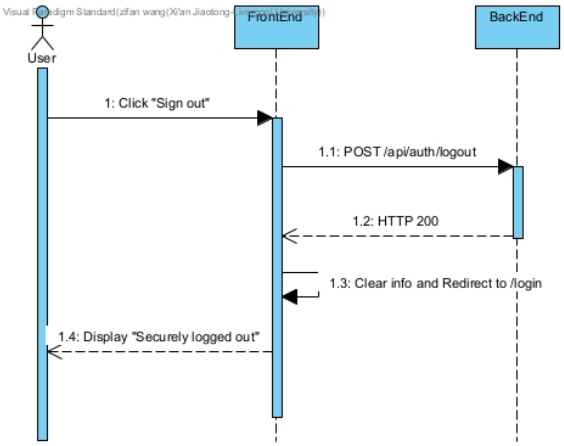


Figure 21: Logout

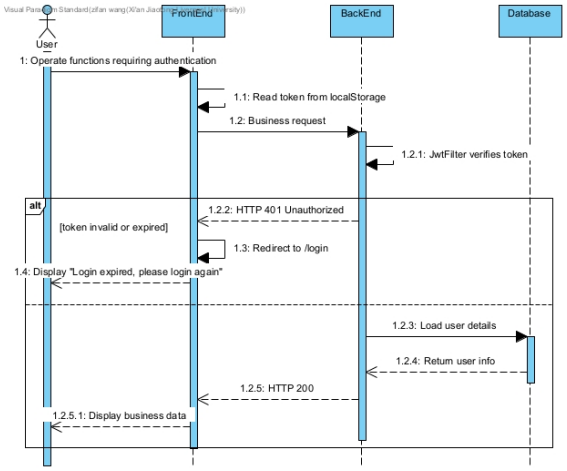


Figure 22: Auth

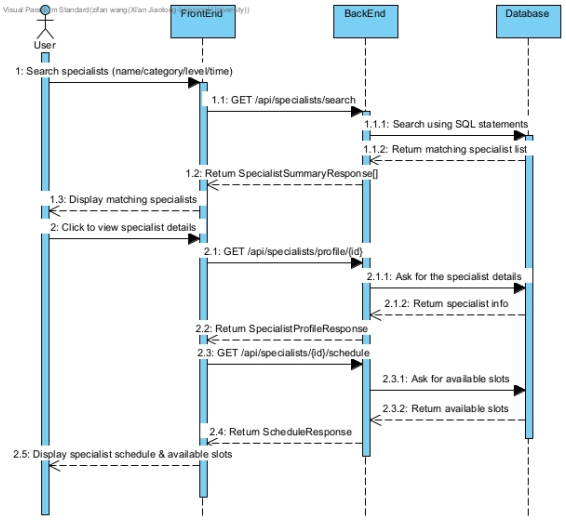


Figure 23: Search & Browse Specialists

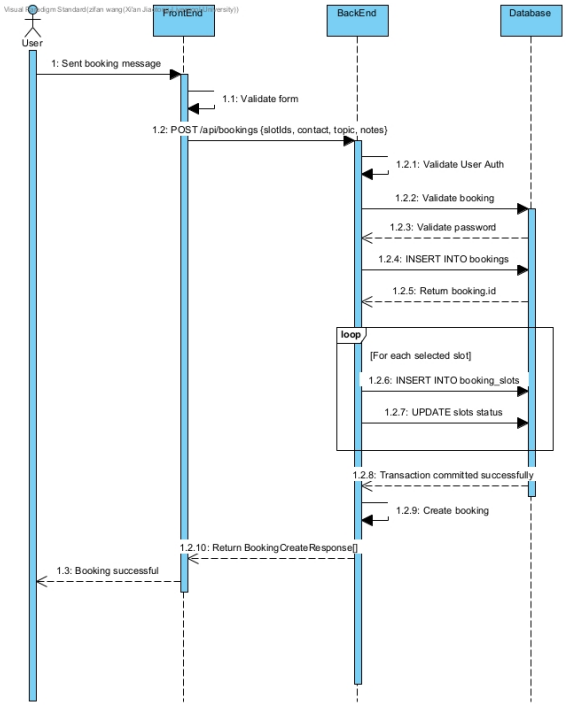


Figure 24: Submit Booking

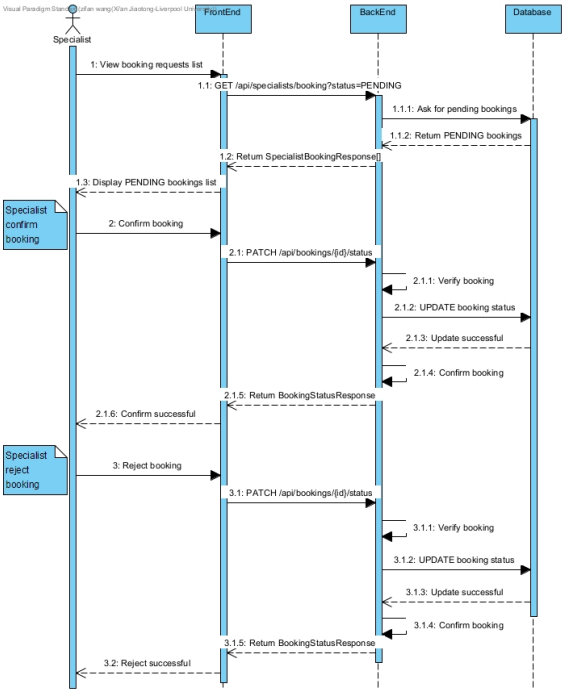


Figure 25: Status Trans & Answer booking

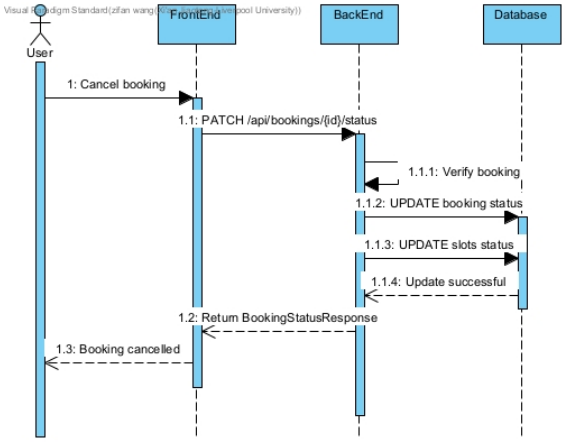


Figure 26: Cancel booking

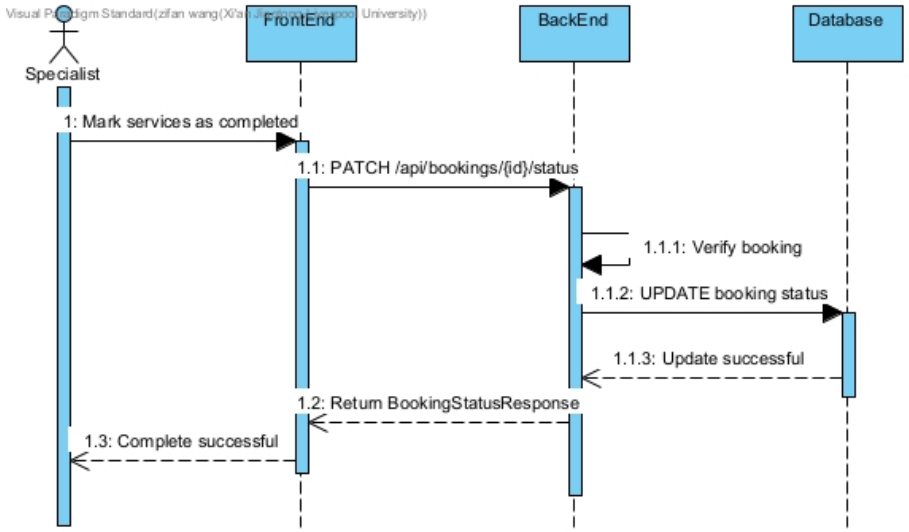


Figure 27: Complete booking

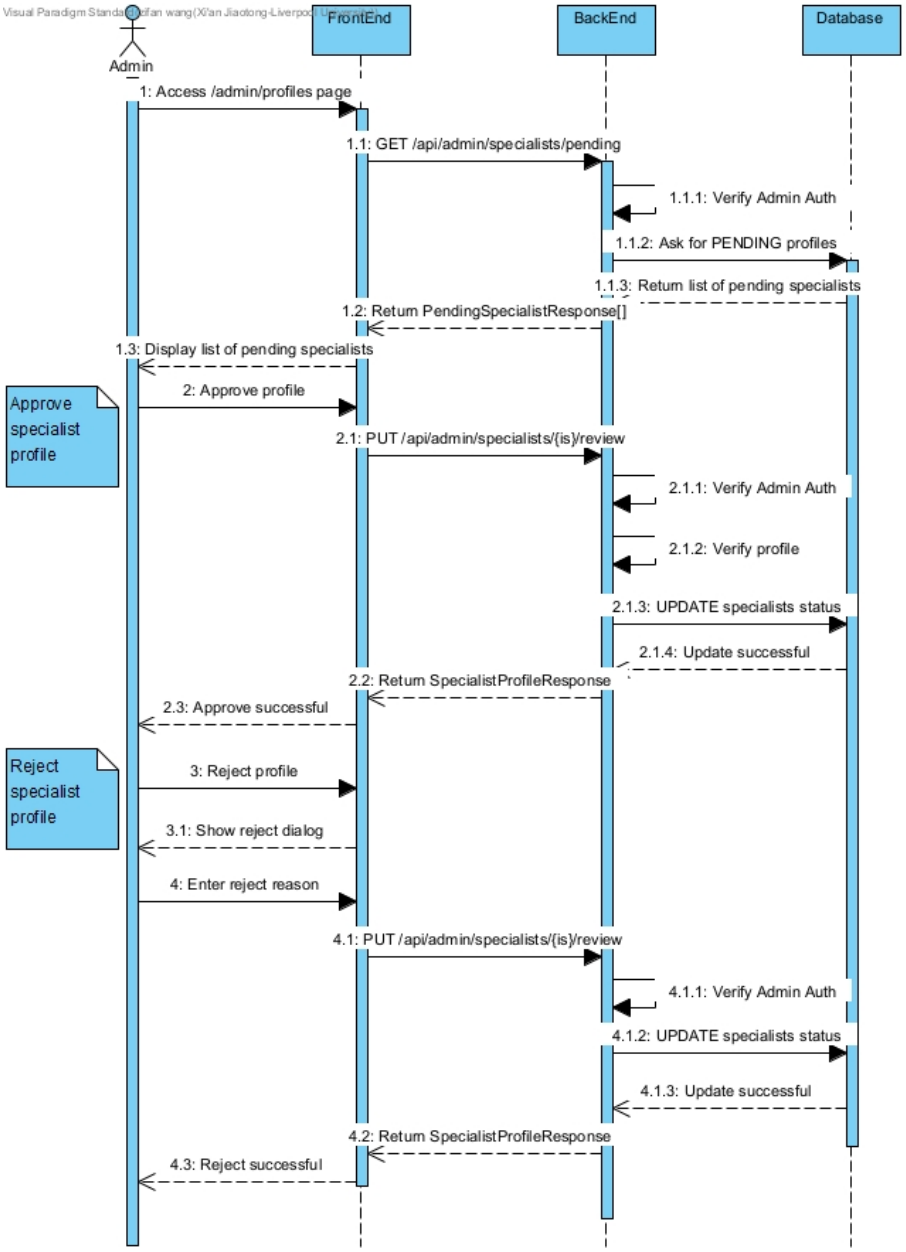


Figure 28: Admin

F Sprint

Sprint 1 · Foundation and Frame					
Sprint Duration	Start Date: 30th March 2026 End Date: 12th April 2026				
Sprint Goal	**1. Backend setup:** Initialize the Spring Boot framework; design key modules, design API contract. **2. Frontend setup:** design website UI prototype **wireframes; chosen frontend solution.** **3. DevOps:** Github repo setup; pull request template and rules setup.				
	Mon	Tue	Wed	Tur	Fri
Week 5	Set up the GitHub repository and invite collaborators		Initialize the Spring Boot framework		Team Discussion
Week 6	Select the frontend solution Design website UI prototype			Testing and Debug	Team Discussion
Task Distribution	**1. Backend setup:** Kan Liu, Zifan Wang, Na Li, Xilai Bu. **2. Frontend setup:** Qi Cao, Shengxiang Zhu. **3. DevOps:** Jintong Zhang, Yuanhao Cheng. **4.Testing and debug:** Yuanhao Cheng				

(a) Sprint 1

Sprint 2 · Role Seperation & Core implement					
Sprint Duration	Start Date: 13th April 2026 End Date: 26th April 2026				
Sprint Goal	1. Confirm the API contract 2. Develop the login and registration module 3. Develop the customer dashboard module 4. Develop the specialist dashboard module 5. Develop the admin panel module				
	Mon	Tue	Wed	Tur	Fri
Week 7	Confirm the API contract	Implement the backend logic for the login and registration module			
	Implement the backend logic for the customer dashboard module				
	Implement the backend logic for the admin panel module				
Week 8	Develop login and registration module frontend and integrate with backend APIs Develop the customer dashboard module frontend and integrate with backend APIs Develop the admin panel module frontend and integrate with backend APIs			Testing and Debug	Daily scrum
Task Distribution	**1. API contract**: Zifan Wang, Jintong Zhang **2. Login and registration module:** Kan Liu, **3. Customer dashboard module**: Na Li, Xilai Bu **4. Specialist dashboard module**: Yuanhao Cheng, Shengxiang Zhu **5. Develop the admin panel module:** Qi Cao 6. **Testing and debug:** Yuanhao Cheng				

(b) Sprint 2

Sprint 3 · Completion & Deployment					
Sprint Duration	Start Date: 27th April 2026 End Date: 8th May 2026				
Sprint Goal	1. Implement Review system 2. Implement announcement feature 3. Testing and Debug 4. Containerization and Deployment				
	Mon	Tue	Wed	Tur	Fri
Week 9	implement reviews and announcements			Testing connectivity	
	Test and Debug				
Week 10	Containerization and Deployment			Traversing User Journey	Sprint Review
Task Distribution	**1. Implement Review system:** Xilai Bu, Shengxiang Zhu **2. Implement announcement feature**: Qi Cao **3. Testing and Debug:** Yuanhao Cheng, Na Li **4. Containerization and Deployment:** Zifan Wang, Jintong Zhang				

(c) Sprint 3

Figure 29: Overview of the Three Development Sprints

G Test Result Screenshots

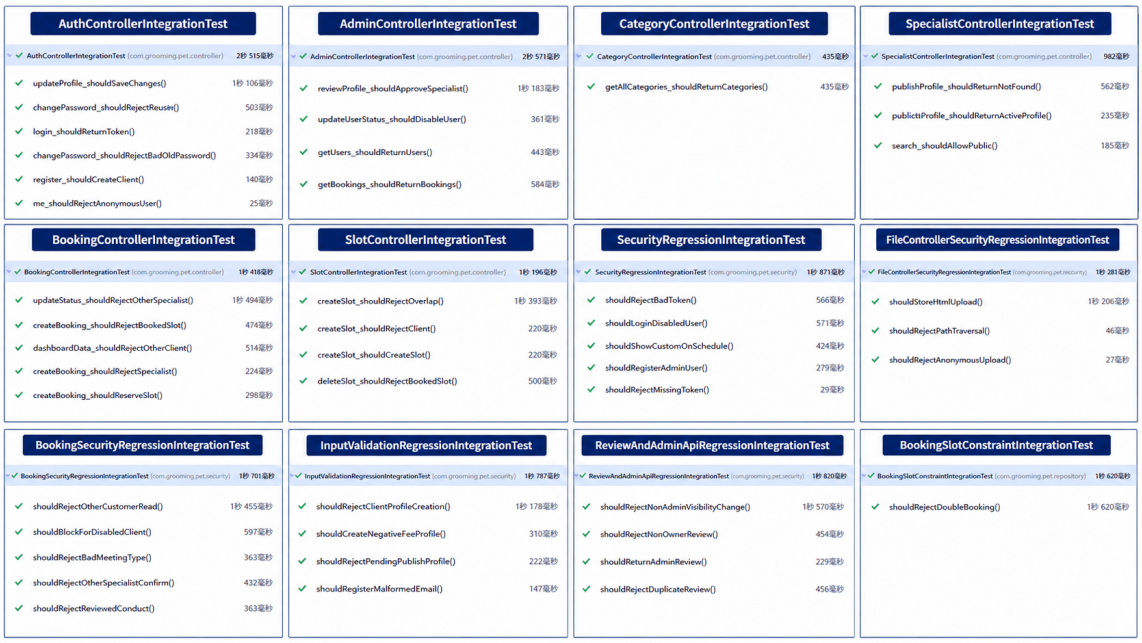


Figure 30: Successful JUnit test results for service-layer unit tests

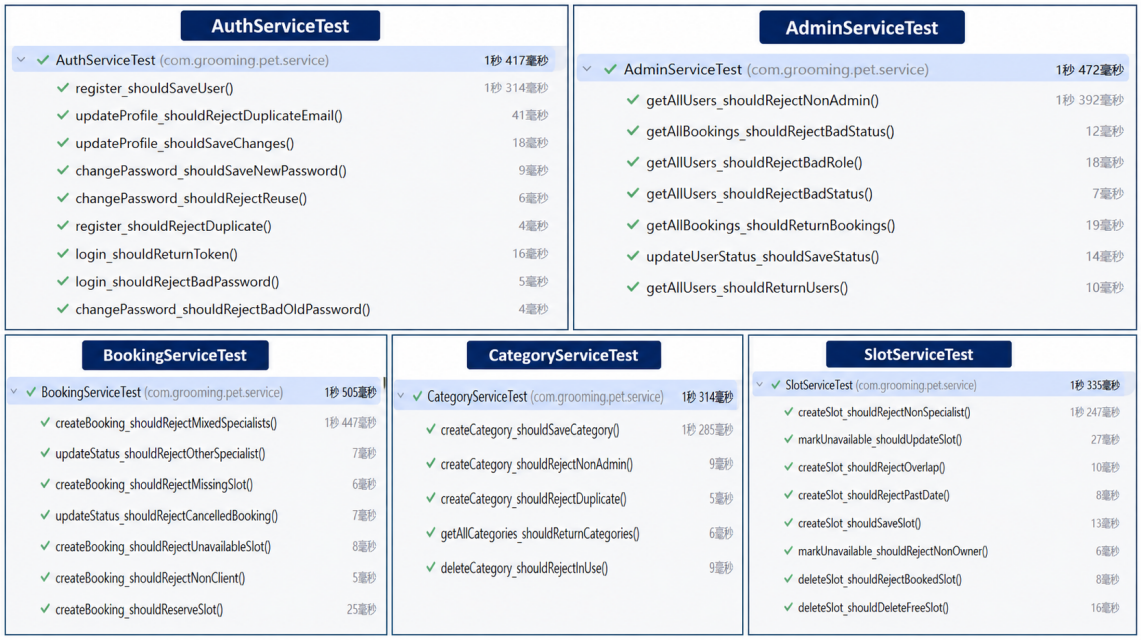


Figure 31: Successful integration test results for controller, security, regression, and repository tests

Figure 32: Successful additional Spring test results for Web MVC slice testing and application context loading

H Defect Tracking and Improvement Before First Release

Table 2 summarises the defects that were identified through the layered testing strategy (unit, integration, and security-regression tests). Every defect was accompanied by a dedicated automated regression test to ensure it could not reappear unnoticed.

Table 2: Defect Tracking Summary

No.	Description	Root Cause	Severity	Solution	Status
D-001	Specialist fee accepted negative values (e.g. -10)	Missing `@Positive` annotation on the DTO `fee` field; no defensive check in the service layer	High	Added `@Positive` annotation and service-layer guard `if (fee ≤ 0) throw`	Fix proposed; pending implementation
D-002	Admin review page could not load – frontend called `GET /api/admin/reviews` while backend exposed `GET /api/reviews/admin/all`	No formal API contract review between front-end and back-end teams; conflicting REST path conventions	Medium	Added `GET /api/admin/reviews` in `AdminController` to align with the front-end expectation	Fixed & verified
D-003	Registration accepted invalid email formats (e.g. `"not-an-email"`)	`RegisterRequest.email` had only `@NotBlank`, missing `@Email`	Medium	Added `@Email(message = "Invalid email format")` annotation	Fix proposed; pending implementation
D-004	Booking confirmation accepted arbitrary `meetingType` (e.g. `"HACKED"`) and dangerous `meetingInfo` (e.g. XSS vector)	No allowed-value constraint on `meetingType`; no content sanitisation	High	Introduced `validateConfirmMeetingDetails()` to restrict `meetingType` to {`ONLINE`, `OFFLINE`} and limit `meetingInfo` length	Closed – all regression tests pass

Defect D-004 illustrates the full lifecycle of a high-severity issue that was identified, analysed, fixed, and closed entirely within the testing framework. The security-regression test `BookingSecurityRegressionIntegrationTest` detected that a `meetingType` value of `"HACKED"` resulted in a 200 OK response instead of the expected 400 Bad Request. Root-cause analysis traced the fault to the `BookingService.confirm()` branch, where the request fields were stored verbatim without validation. The fix introduced a dedicated validation method, `validateConfirmMeetingDetails()`, backed by a mutable-constant set `ALLOWED_MEETING_TYPES`. After the fix, the same regression test verified three green-line criteria: (1) the endpoint returns 400; (2) the booking status remains `PENDING` in the database; (3) the malicious `meetingType` and `meetingInfo` are not persisted. This defect was closed with full confidence.

Table 3 gives a quantitative overview of the defect management effort.

Table 3: Defect Statistics

Metric	Value
Total defects discovered	4
By severity: High / Medium / Low	2 / 2 / 0
Closed (fully verified)	1 (D-004)
Fixed and verified	1 (D-002)
Fix proposed, awaiting implementation	2 (D-001, D-003)
Defects covered by automated regression tests	4 / 4 (100%)

In addition to the detailed fixes, the defect tracking process triggered broader improvements to the development workflow. The API-contract mismatch (D-002) prompted the team to adopt a formal rule: every cross-team API change must be documented in a shared specification file, reviewed by both front-end and back-end leads, and enforced by an automated contract test. The input-validation gaps (D-001,

D-003, D-004) led to a systematic review of all DTO classes, resulting in the addition of `@Positive`, `@Email`, and `@Pattern` annotations wherever applicable.